

A Non-Corrective Poetry Editor for Marking Ambiguous Fragments in Creative Writing

Dmitry Neledva
Independent Project
Vladimir, Russia
dima.neldeva@icloud.com

Abstract

Standard text editors are designed to correct mistakes, but creative writing often includes unusual words, rhythm shifts, and non-standard forms that may be intentional. This paper presents a prototype of a poetry editor that marks ambiguous fragments instead of automatically correcting them. The editor allows the author to classify each fragment as a mistake, an authorial device, an ignored comment, a postponed decision, or a personal dictionary entry. The system also includes a local authorial corpus, a personal dictionary, and a rhythm map based on line-level syllabic structure. The goal of the project is to support creative editing by turning possible errors into reviewable authorial decisions rather than automatic corrections.

Keywords

creative writing, poetry editor, authorial text, digital editing, ambiguous fragments, user dictionary

1 Introduction

Standard text editors are usually designed to correct mistakes. They work well with ordinary texts, where a spelling error, an unusual word form, or a strange punctuation mark is usually treated as something that should be fixed. However, creative writing works differently. In poetry, prose, lyrics, and other artistic texts, unusual fragments may be intentional. A strange word can be part of the author's vocabulary, a broken rhythm can create tension, and a non-standard line break can be part of the text structure.

This creates a problem for digital editing tools. If the editor treats every unusual fragment as an error, it may push the author toward a more neutral and predictable form of writing. The text may become technically cleaner, but it can also lose part of its individual style.

This paper presents a prototype of a non-corrective poetry editor. The editor does not automatically rewrite the text or decide what is right. Instead, it marks ambiguous fragments and allows the author to classify them manually. A fragment can be accepted as a mistake, kept as an authorial device, ignored, postponed, or added to a personal dictionary.

The main idea is to transform possible errors into reviewable authorial decisions. The editor is designed as a practical tool for working with authorial texts, ambiguous words, rhythm shifts, and local text archives.

This paper makes three small practical contributions:

- it defines the ambiguous fragment as the main unit of a creative text editor;
- it proposes a simple decision model for classifying fragments without automatic correction;

- it describes a working prototype with a personal dictionary, local corpus, comment list, and rhythm map.

2 Problem

The main problem is the difference between an error and an intentional deviation. In ordinary writing, this difference is usually simple: a non-standard word form is often a mistake. In creative writing, the same fragment can have another role. It can express voice, rhythm, irony, emotional tension, or the author's personal style.

A standard editor can detect formal deviation, but it cannot know whether the deviation is accidental or intentional. Because of this, automatic correction becomes risky for creative texts. The tool may correct not only mistakes, but also fragments that the author wanted to keep.

Table 1: Ambiguous fragments in creative texts

Fragment	Standard editor	Creative text
Unusual word	Error	Authorial vocabulary
Broken rhythm	Problem	Artistic effect
Strange punctuation	Mistake	Voice or rhythm
Line break	Formatting issue	Poetic structure
Non-standard form	Incorrect form	Stylistic device

The proposed editor is based on a different approach. It does not treat every detected fragment as an error. Instead, it treats it as an ambiguous fragment that requires an authorial decision. This means that the editor's task is not to correct the text automatically, but to help the author notice, review, and classify questionable places.

This approach is especially important for poetry, where form is not only a technical container for meaning. Line length, pauses, rhythm shifts, strange words, and breaks can all be part of the artistic structure. Therefore, a creative text editor should not only check correctness. It should also preserve the author's control over the final form of the text.

3 Editor Design

The editor is based on a simple idea: the main unit of work is not an error, but an ambiguous fragment. An ambiguous fragment is a word, line, or text segment that may require attention, but cannot be automatically classified as wrong. The editor marks such fragments and lets the author decide what they mean.

The key design decision is to separate detection from correction. The system may detect a suspicious fragment, but the author decides how it should be interpreted. This makes the tool useful for creative writing, where unusual forms may be intentional.

Table 2: Main modules of the proposed editor

Module	Purpose
Text editor	Writing and editing poems or other creative texts
Comments list	Collecting all ambiguous fragments in one place
Orthography module	Detecting suspicious or unknown words
Rhythm module	Showing line-level rhythm shifts
User dictionary	Storing accepted authorial words
Local corpus	Storing texts, notes, and drafts

Each comment contains basic information about the fragment: its position, type, severity, message, and current status. This structure allows the editor to separate detection from correction. The system can detect a questionable place, but the final classification remains manual.

Table 3: Structure of an editor comment

Field	Meaning
Line	Position of the fragment in the text
Fragment	Word, line, or text segment
Type	Orthography, rhythm, or text mechanics
Severity	Weak, medium, or strong deviation
Message	Short explanation of the comment
Status	Current authorial decision

The most important part of the design is the decision model. The editor does not force the author to accept a correction. Instead, it offers several possible actions. This allows the same fragment to be treated differently depending on context.

Table 4: Authorial decisions for ambiguous fragments

Decision	Meaning
Correct	The author accepts the fragment as a mistake
Keep	The fragment is intentional
Ignore	The comment is not relevant
Later	The author postpones the decision
Dictionary	The word is added to the personal dictionary

For example, an unknown word can be corrected if it is a real typo, ignored if the comment is not important, or added to the dictionary if it belongs to the author’s vocabulary. A rhythm shift can also be kept if the author sees it as part of the poem’s structure.

The editor therefore works as a decision-support tool. It does not replace the author’s judgment, but organizes the revision process. The author sees all questionable places, reviews them one by one, and decides what should happen to each fragment.

4 Prototype Implementation

The prototype was implemented as a local poetry editor with several working modules. The interface allows the author to write or import a text, run a check, review comments, work with rhythm information, and save texts to a local corpus.

The editor does not use automatic correction as its main action. Instead, it produces a list of comments. Each comment points to a fragment that may require attention. The author can then decide how to treat the fragment. This keeps the editing process under the author’s control.

The prototype includes two main types of detection. The orthography module finds suspicious or unknown words. These words are not treated as final errors, because in creative texts an unknown word can be a neologism, a personal expression, or an intentional distortion. The rhythm module builds a line-level syllabic profile and marks lines that differ from the internal rhythm of the text.

4.1 Local Data Storage

The editor stores data locally. This makes the prototype simple and independent from external services. Two local files are used: one for the author’s corpus and one for the personal dictionary.

Table 5: Local data stored by the editor

File	Stored data	Purpose
author_corpus.json	Texts, dates, notes	Personal archive
user_dictionary.json	Accepted words	Authorial vocabulary

In the current prototype, the corpus entry contains an identifier, title, creation date, text body, and notes. The dictionary stores accepted authorial words as a list of strings. These structures are intentionally simple, because the prototype is focused on the editing workflow rather than complex data management.

The corpus stores saved texts with metadata such as title, creation date, text body, and notes. The user dictionary stores words that should not be marked as spelling errors in future checks. This is important for creative writing, because the author’s vocabulary may include unusual or non-standard words.

4.2 Orthography Module

The orthography module detects words that look suspicious from the point of view of a standard dictionary. However, the module does not force a correction. It only creates a comment. The author can correct the word, ignore the comment, postpone the decision, or add the word to the personal dictionary.

This behavior is important for authorial texts. For example, a standard spellchecker may treat a new word as an error, while the author may see it as part of the style. The dictionary function allows the editor to adapt to the author’s language over time.

4.3 Rhythm Module

The rhythm module works with the line structure of the poem. It counts syllables in each line and compares the result with the internal average of the text. Delta is calculated as the difference between the syllable count of a line and the internal average syllable count of the text.

The goal is not to perform a full literary analysis of rhythm. The goal is to show where the text changes its line-level rhythm. The module marks lines as normal, moderately deviating, or strongly deviating. This helps the author notice long lines, short lines, sudden rhythm shifts, and unstable areas of the text.

Line	Map	Syllables	Delta	Mark
12	#####	11	+7.5	!
13	###	3	-0.5	.
14	##	2	-1.5	.
20	#	1	-2.5	*
21	#	1	-2.5	*
24	#####	9	+5.5	!
26	#####	6	+2.5	*

Figure 1: Example of a line-level rhythm map showing syllable count, deviation from the internal average, and deviation mark.

In this map, the bar length represents the number of syllables in the line. The delta value shows the difference between the line and the internal average. The mark shows the strength of the deviation. A strong deviation does not mean that the line is bad. It only means that the line differs from the general rhythm profile and may require the author's attention.

4.4 Comment List

All detected fragments are collected in the comment list. This list works as the main editing workspace. Instead of searching through the whole text manually, the author can move through comments one by one and decide what to do with each fragment.

The comment list combines different types of remarks: orthographic comments, rhythm comments, and technical comments. This makes the revision process more organized. The author sees not only separate mistakes, but also the overall state of the text.

4.5 Corpus Module

The corpus module allows the author to save texts locally and return to them later. This turns the editor into a small personal archive, not only a checking tool. The saved corpus can later be extended with versions, decisions, and additional metadata.

The current implementation is simple, but it already supports the main idea of the project: creative editing should not only correct a text, but also preserve the author's work, vocabulary, and decisions.

5 Example Use Case

This section shows a simple use case of the editor. The goal is not to demonstrate a full literary analysis, but to show how the prototype supports the author's revision process.

The author writes or imports a poem and runs the check. The editor detects several ambiguous fragments: suspicious words, rhythm shifts, and technical features of the text. These fragments are collected in the comments list. The author then reviews them one by one and chooses the appropriate decision for each case.

Table 6: Example author workflow

Step	Editor action	Author action
1	Loads or receives the text	Starts working with the poem
2	Detects ambiguous fragments	Reviews the comment list
3	Shows suspicious words	Corrects or keeps them
4	Shows rhythm shifts	Decides whether they are intentional
5	Offers dictionary action	Adds authorial words if needed
6	Saves the text locally	Keeps the text in the corpus

The same type of fragment can lead to different decisions. For example, an unknown word can be a typo in one text and an authorial word in another. A short line can be accidental in one case and structurally important in another. Because of this, the editor does not apply one universal rule to all fragments.

Table 7: Example decisions for ambiguous fragments

Fragment	Comment type	Author decision
<i>neiroresursy</i>	Orthography	Add to dictionary
Possible typo	Orthography	Correct
Very long line	Rhythm	Keep as intentional
Very short line	Rhythm	Review later
Unusual break	Text mechanics	Ignore

The word *neiroresursy* is used here as an example of an authorial word. A standard dictionary may treat it as suspicious, but the author may keep it as part of personal vocabulary. In this case, adding the word to the dictionary is more appropriate than correcting it automatically.

This workflow shows the main difference between the proposed editor and a standard correction tool. The system does not try to decide what the final text should be. It only makes questionable places visible and stores the author's response to them.

As a result, the revision process becomes more structured. The author can see all detected fragments, classify them, and save the text together with personal vocabulary and notes. This is especially useful for creative writing, where unusual forms should not always be removed.

6 Limitations and Future Work

The current prototype has several limitations. First, the editor does not understand the meaning of the poem. It can detect suspicious

words, unusual line lengths, and rhythm shifts, but it cannot decide whether they are artistically successful. This decision must remain with the author.

Second, the rhythm module is based on a simple syllabic profile. It counts syllables and compares lines with the internal average of the text. This approach is useful for showing visible rhythm shifts, but it is not a full prosodic analysis. It does not analyze stress, intonation, rhyme, pauses, or sound patterns.

Third, the orthography module can mark authorial words as suspicious. This is expected behavior, because creative texts often contain neologisms, distorted words, or unusual forms. The personal dictionary partly solves this problem, but the system still requires manual decisions from the author.

Fourth, the current corpus is stored locally and has a simple structure. It works as a personal archive, but it does not yet support version history, advanced search, or detailed statistics about authorial decisions.

Fifth, the current paper does not include a user study. Therefore, it does not claim that the editor improves creative writing. It only presents the design logic and prototype workflow.

Future work will focus on improving the decision model and the corpus. The editor can be extended with text versioning, export of annotated texts, search through comments, and statistics about repeated authorial devices. Another possible direction is to store not only the final text, but also the history of decisions: which fragments were corrected, ignored, postponed, or added to the dictionary.

A later version may also include optional AI-assisted suggestions. However, such suggestions should not become automatic corrections. The main principle of the editor should remain the same: the system may show possible problems, but the final decision belongs to the author.

7 Conclusion

This paper presented a prototype of a non-corrective poetry editor for working with ambiguous fragments in creative texts. The main idea of the project is simple: a creative editor should not automatically decide whether an unusual word, line break, or rhythm shift is wrong. Instead, it should make such fragments visible and leave the final decision to the author.

The proposed editor is built around comments and authorial decisions. A detected fragment can be corrected, kept as intentional, ignored, postponed, or added to the personal dictionary. This makes the editor different from standard correction tools, where unusual language is often treated only as an error.

The prototype also includes a local corpus, a user dictionary, an orthography module, and a rhythm map based on line-level syllabic structure. These elements support the main goal of the project: to help the author revise creative texts without losing control over style and form.

The current version is limited and does not perform full literary analysis. However, it demonstrates a practical approach to creative editing: the editor does not replace the author's judgment, but organizes the process of noticing, reviewing, and classifying questionable places in the text.

The prototype demonstrates that a creative editor can be designed around authorial decisions rather than automatic corrections.

References

- [1] LanguageTool. 2026. LanguageTool: AI-based spelling, style, and grammar checker. Retrieved June 13, 2026 from <https://languagetool.org/>
- [2] Grammarly. 2026. Grammarly writing assistant. Retrieved June 13, 2026 from <https://www.grammarly.com/>
- [3] Donald A. Norman. 2013. *The Design of Everyday Things: Revised and Expanded Edition*. Basic Books.
- [4] Ben Shneiderman. 2022. *Human-Centered AI*. Oxford University Press.
- [5] Lev Manovich. 2020. *Cultural Analytics*. MIT Press.
- [6] Matthew L. Jockers. 2013. *Macroanalysis: Digital Methods and Literary History*. University of Illinois Press.
- [7] Association for Computing Machinery. 2026. ACM Primary Article Template. Retrieved June 13, 2026 from <https://www.acm.org/publications/proceedings-template>